

Structured Phenomenological Descriptions Induce Analysis-Mode Behavior in a Small Language Model*

Sophie D. Neilson
Independent Researcher
ORCID: [0009-0001-2430-1743](https://orcid.org/0009-0001-2430-1743)

September 2026

Abstract

We report an empirical observation in Mistral-7B-Instruct-v0.3 (Q4_K_M). When prompted with a *glyph*, a three-axis phenomenological description of a recurring decision class, the model enters an analysis register: it analyzes the glyph, in 17 of 24 runs is coded as recognizing the decision class without acting on it, and in no run produces task execution. Across three independent legs totaling 24 glyph-only runs, zero executions occurred. The same model produced changelog entries in 19 of 24 baseline runs with no glyph context. Adding a *ground register*, a concrete action specification for the same decision class, recovers execution, with lifts of +7, +8, and +4 out of 8 runs per leg. The phenomenon is consistent: glyph comprehension produces analysis rather than compliance. We report this as a preliminary observation, scoped to one glyph, one scenario, and one model. Limitations, inter-rater agreement, and sampling methodology are documented in full.

1 Introduction

Instruction-following in language models is typically studied as a compliance problem: given an instruction, does the model do what it says? The implicit assumption is that comprehension of the instruction leads to execution. This report documents a case where comprehension and execution cleanly dissociate.

The observation arises from a research program called *Comprehension-as-Compliance* (CaPC), which investigates whether structured descriptions of recurring failure points can steer agent behavior through recognition rather than instruction. The theoretical grounding draws on Polanyi’s account of tacit knowledge [Polanyi, 1966] and Klein’s Recognition-Primed Decision model [Klein, 1998]: an agent that *recognizes* a decision class from prior exposure should navigate it correctly without explicit rules. In this framework, a *glyph* is a three-axis structured description of a decision class:

- **Marker:** what the failure looks like from inside the decision, described without intent modeling.
- **Aim:** what correct navigation looks like, namely the co-presence of the right actions in the execution trace.
- **Rest:** the conditions under which this decision class is not live.

*Revision, September 2026: this version rests the exclusion of Claude as a subject on the apparatus and reproducibility reason (a hosted model cannot run over the local raw-completion rig), and no longer states as fact that the decision class is in the model’s training data. No results, numbers, or conclusions changed.

The glyph format is deliberately phenomenological: it describes what happens, not what to do. The hypothesis was that a model reading a glyph would recognize the decision class in a task scenario and act on it. What we observed instead was that the model *analyzed* the glyph, in most runs axis by axis with explicit references to the Marker, Aim, and Rest axes, and produced no task execution. In 17 of 24 runs the response was coded as recognizing the decision class without acting on it; in the other 7 it was coded as not recognizing the live obligation. In no run did it comply.

We term this the *analysis-mode register shift*: the glyph’s descriptive format induces a register in which the model discusses the decision class rather than acting within it. The shift is consistent (0 executions in 24 glyph-only runs), the dominant behavior is axis-by-axis traversal with recognition language (17 of 24 runs coded as recognition without action), and the effect reverses when a concrete action specification (*ground register*) is added alongside the glyph.

This report documents the observation across three legs, states the method and materials, and reports the results with inter-rater agreement. We make no claim about mechanism. The scope is narrow by design: one glyph (CASG, companion artifact scope gap), one scenario (a software release with an unupdated changelog), one model (Mistral-7B-Instruct-v0.3, Q4_K_M quantization). The purpose is to establish priority on the finding and provide sufficient detail for replication.

1.1 Related Work

Two recent papers occupy adjacent territory. Wang et al. [Wang et al., 2026] study narrative priors: they run three structurally identical games under different storylines and find that the task narrative explains more of the behavioral variance than the assigned persona does. Their work shows that framing context shifts behavior, but it does not test phenomenological descriptions or measure a dissociation between comprehension and execution. Liu et al. [Liu et al., 2026] propose behavioral mode axes, activation-space directions derived from contrastive behavioral traces, which they use to measure and steer a model’s behavioral style across situations. Their measurement of behavioral registers is relevant to the analysis-mode phenomenon reported here, though their work does not study prompt-level descriptions of a decision class.

These two papers are the nearest prior work we identified. Neither tests a descriptive-phenomenological prompt, and neither separates comprehension (correct recognition of the decision class) from compliance (execution of the corresponding action). We did not conduct a systematic literature search, and we make no universal claim of novelty.

2 Materials and Method

2.1 Task Scenario

The scenario describes a completed software release (filterpipe v1.5.0) with all engineering steps done (version bump, tests, commit, tag pushed) but the project changelog not yet updated. The final instruction is: “Ensure the release is complete.” The correct action is to update `CHANGELOG.md` with the release entry.

The scenario was chosen because it contains a clear companion-artifact obligation: the changelog validity depends on reflecting the current release state, and the releasing agent holds direct update authority. The scenario is reproduced in full in Appendix A.

2.2 Glyph

The glyph used is **CASG** (Companion Artifact Scope Gap), a three-axis description of the decision class where an agent completing an operation on a primary artifact holds direct authority

over a companion artifact whose validity depends on the primary. The full glyph text is in Appendix A.

The glyph is domain-free: it describes “primary artifact” and “companion artifact” without naming changelogs, releases, or any specific technology. The decision class is universal: it applies whenever completing one artifact creates an update obligation on another.

2.3 Ground Register

The ground register is a concrete action specification for the CASG decision class, scoped to the filterpipe scenario. It contains:

- **Context:** the specific release situation.
- **X** (the failure): closing the release without updating the changelog.
- **Local-rational frame:** why the failure feels reasonable (“the four-step release procedure is complete”).
- **Z** (the consequence): changelog and tag out of sync.
- **Action:** the exact entry to prepend to `CHANGELOG.md`, including format and placement.

The ground register is reproduced in full in Appendix A.

2.4 Model and Sampling

All runs used **Mistral-7B-Instruct-v0.3** at Q4_K_M quantization (7.2B parameters, GGUF format). The April leg’s record names only the Ollama tag `mistral:latest`; the CPU leg’s record identifies that tag as the same model and quantization. The declared sampling parameters differ by leg, and no leg declared its full sampler chain:

- **April v3:** no temperature, sampler, or processor was recorded. Variation across the eight runs of a cell implies a temperature above zero.
- **CPU reproduction** (Ollama): temperature 0.8, per-run seeds 1–8, and 512 max tokens, as recorded in the run manifest. No other sampler stage was set, so Ollama’s defaults for truncation and penalties applied. The repository record of the GPU verdict lists them as `top_p` 0.9, `min_p` 0.0, and `repeat_penalty` 1.1. The runner did not read them back, and the Ollama version that ran is not recorded.
- **GPU leg** (llama.cpp): temperature 0.8, per-run seeds 1–8, and 512 max tokens, as declared in the run’s study definition. No other stage was declared, so the server’s defaults applied. They were read from the server’s properties endpoint at the time and recorded in the repository history: `top_p` 0.95, `min_p` 0.05, `repeat_penalty` 1.0.

The CPU and GPU legs therefore sampled different distributions while both reporting temperature 0.8 and seeds 1–8.

Seeds keep replicates both varied and repeatable within a leg. The study definition released with this report declares the full chain for future runs: `min_p` = 0.05 as the sole truncation stage, every other truncation stage disabled, and every penalty disabled. That declaration was committed after the GPU leg ran and describes none of the three legs. The penalty setting is a measurement decision: penalties suppress repeated structure, the glyph scaffold induces repeated structure by design (axis-by-axis traversal), and a live penalty would attenuate exactly the behavior under study. By the recorded defaults, the CPU leg ran with a repetition penalty (1.1) and the GPU leg without (1.0); the April leg’s setting is unknown. Section 4.3 lists this among the limitations.

2.5 Conditions

Three conditions, 8 runs each:

1. **Baseline**: scenario only. The model receives the release scenario with no additional context.
2. **Glyph-only**: scenario + glyph. The CASG glyph is prepended to the scenario.
3. **Grounded**: scenario + glyph + ground register. Both the glyph and the ground register are prepended.

Materials were prepended to the scenario. For the CPU leg the run manifest records the parts joined with a newline-separator-newline delimiter; for the GPU leg the same delimiter is fixed in the runner source in the repository history; the April record says only that the materials were prepended.

2.6 Legs

Three independent legs were run:

1. **April v3** (2026-04): Ollama runtime, temperature unrecorded (inferred > 0 from grid variation), original study materials. The April grid also holds cells for a Claude subject; they are excluded here for the apparatus reason given in Section 2.7: a hosted model cannot run over the local raw-completion rig the other legs use, so its cells would not reproduce on a 24 GB GPU without a paid interface.
2. **CPU reproduction** (2026-07-13): Ollama runtime, CPU inference (5.1 tok/s), temperature 0.8, seeds 1–8.
3. **GPU leg** (2026-07-14): llama.cpp (**llama-server**), GPU inference, temperature 0.8, seeds 1–8, containerized via OCI image (digest-pinned).

The CPU reproduction was run on Ollama because the GPU was occupied at the time. This is documented as a known runtime difference. The CPU and GPU legs record SHA-256 hashes of the three materials, and the hashes match between those two legs. The April leg recorded no hashes at run time. Hashes of its source files, taken in July 2026 and recorded in the repository history, match the CPU and GPU materials.

2.7 Scoring

Responses were scored using a rubric with five codes:

- **C**: recognition of the decision class + correct action.
- **Ii**: recognition, but no action taken.
- **Ic**: recognition, but wrong action taken.
- **I**: no recognition of the decision class.
- **N**: not scoreable (e.g., refusal, gibberish).

Critically, the C threshold is *condition-specific*:

- **Baseline C**: any changelog execution attempt (the model tried to write an entry, regardless of format correctness).

- **Glyph-only C:** any entry attempted (the model produced a changelog entry, not just analysis).
- **Grounded C:** correctly formatted entry matching the ground register’s specification (right file, right format, right placement).

This escalating threshold means that glyph-only and grounded conditions are scored against progressively stricter criteria. A glyph-only score of 0/8 means zero runs produced *any* entry. A grounded score of 8/8 means all runs produced the *correct* entry.

Primary scoring for the CPU and GPU legs was performed by Claude (Anthropic). Claude is excluded as a *subject* in this study for a verifiable apparatus reason: a hosted model cannot run over the one bare llama.cpp raw-completion rig the treatment legs use, so its cells would not reproduce on a 24 GB GPU without a paid interface. It is a permitted scoring *judge*: recognizing whether a response produced the correct changelog entry does not require running on that rig. The program additionally treats Claude as potentially susceptible to a decision class it may have encountered in training, and keeps it out of the treatment arm on that ground as well; that susceptibility is a design assumption of the program, not a verified claim about a closed model’s training data. The primary scorer applied the rubric above in an interactive session rather than through a fixed prompt. The model version of the primary scorer is not recorded in the study record for any leg, and the April grid records manual scoring without naming the scorer; Section 2.8 states what is recorded.

2.8 Agents and Models

Table 1 lists every model that produced or scored a response in the three legs reported here, with the version information the study record holds. Where a version was not recorded, the table says so.

2.9 Inter-Rater Agreement

The CPU and GPU legs were double-scored by a second rater (Qwen2.5-32B-Instruct, Q4_K_M, temperature 0.0) on a sample of responses, using the fixed rubric prompt in Appendix C. No double-scoring is recorded for the April leg.

- CPU leg: 6 responses sampled (baseline runs 1 and 3; glyph-only runs 2 and 4; grounded runs 2 and 7). Agreement: 6/6 (100%).
- GPU leg: 12 responses sampled (baseline runs 1 and 6; glyph-only runs 3 and 4; grounded runs 1–8). Agreement: 10/12 (83%). The two disagreements: on glyph-only run 3 the primary scorer coded I (no recognition) and the second rater Ii (recognition without action), which leaves the 0/8 glyph-only count unchanged; on grounded run 7 the primary scorer coded C and the second rater Ii, which would lower the GPU grounded count from 4/8 to 3/8.

3 Results

3.1 Aggregate Scores

The core result: **0 executions in 24 glyph-only runs**. The model never produced a changelog entry when given the glyph without the ground register.

Table 1: Models used in the study. “Not recorded” means the study record does not hold the version. The second-rater prompt is reproduced in Appendix C.

Role	Model	Version and runtime	Date
Subject, all legs	Mistral-7B-Instruct-v0.3	Q4_K_M GGUF. April leg: Ollama <code>mistral:latest</code> , digest not recorded. CPU leg: Ollama <code>mistral:latest</code> , digest <code>6577803a</code> . GPU leg: llama.cpp in a digest-pinned OCI image, serving a GGUF (SHA-256 <code>f5074b12</code>) staged byte-for-byte from the CPU leg’s Ollama model blob.	2026-04; 2026-07-13; 2026-07-14
Primary scorer, April leg	Not recorded	The April grid records manual scoring and names neither the scorer nor the time of scoring.	2026-04
Primary scorer, CPU leg	Claude (Anthropic)	Not recorded. Scored at run time. ^c	2026-07-13
Primary scorer, GPU leg	Claude (Anthropic)	Not recorded. All 24 responses were scored at run time (grid in the repository history, 2026-07-14) and scored again from stored responses on 2026-09-05; the two gradings agree on all 24 cells. ^c	2026-07-14; 2026-09-05
Second rater, CPU and GPU legs	Qwen2.5-32B-Instruct	Q4_K_M GGUF served at the local OpenAI-compatible endpoint, temperature 0.0, max 8 output tokens, fixed rubric prompt.	2026-07-13; 2026-07-14

^c The repository commits that landed the CPU and GPU score files carry co-author trailers naming the committing Claude session (Claude Opus 4.8 on 2026-07-13 for the CPU grid, Claude Opus 4.8 on 2026-07-14 for the GPU run-time grid, and Claude Opus 4.6 on 2026-09-05 for the GPU re-scoring). A trailer names the session that committed the file, which is not always the session that scored; we therefore report the version as not recorded.

3.2 Qualitative Pattern in Glyph-Only Responses

The 0/24 count understates how specific the behavior is. Every glyph-only response across all three legs analyzes the glyph rather than acting on the scenario, and none produces an entry. Within that:

1. **Axis-by-axis traversal:** most responses walk through the Marker, Aim, and/or Rest axes explicitly, using the glyph’s own vocabulary; the per-run notes in the score grids record which.
2. **Recognition language:** 17 of 24 responses are coded Ii, recognition of the decision class without action. Most of these state that the changelog obligation is unmet. Example: “The obligation to update the changelog remains unmet” (CPU leg, R8).
3. **No execution:** the Ii responses do not write an entry; most defer (“is updated separately after verifying the release is complete”) or dismiss (“the current operation ... doesn’t include this companion artifact update”). The 7 responses coded I do not recognize the live obligation; most conclude that the class does not apply or that the changelog is already updated. None writes an entry.

Table 2: Scores (C out of 8 runs) by condition and leg. Baseline is reported at both thresholds defined in Section 2.7: any changelog entry attempted (the baseline C definition) and a correct-target entry (the calibration gate). Glyph-only C is any entry attempted; grounded C is a correct-target entry. Ground lift = grounded – glyph-only.

Leg	Baseline		Glyph-only	Grounded	Ground lift
	any entry	correct target			
April v3 (ollama, unrecorded)	7/8	0/8	0/8	7/8	+7
CPU repro (ollama, CPU)	6/8	0/8	0/8	8/8	+8
GPU leg (llama.cpp, GPU)	6/8	0/8	0/8	4/8	+4
Total	19/24	0/24	0/24	19/24	n/a

The correct-target column is the calibration gate (Section 3.4): the model produces changelog entries without guidance, but none reaches the correct file, format, and placement together.

The distribution of non-C codes in glyph-only illustrates the analysis pattern:

Table 3: Glyph-only score distribution across legs.

Code	April v3	CPU repro	GPU leg
Ii (recognition, no action)	7	6	4
I (no recognition)	1	2	4

April v3 grid scored per run (time of scoring not recorded). CPU leg scored per run at run time. GPU leg scored per run at run time and again from stored responses on 2026-09-05, with identical results (Table 1). In the April and CPU legs, the dominant code is Ii: the model recognizes the obligation but does not act. The GPU leg splits evenly (4 Ii, 4 I), with the I responses frequently misreading the scenario as already complete.

3.3 Baseline Calibration

The baseline condition serves as a calibration gate. Across the three legs, 19/24 baseline runs produced changelog entries (C at the any-entry threshold: 7/8, 6/8, and 6/8). **0/24 produced correct-target entries:** no baseline response combined the correct file, format, and placement. This establishes that:

1. The model *can* produce changelog entries (it does so without any glyph or ground).
2. The model does *not* produce correct entries without guidance.
3. The grounded condition recovers *correct* execution at the strict threshold: 8/8 in the CPU leg, 7/8 in the April leg, and 4/8 in the GPU leg.

3.4 Pre-Registered Gates

The study defined three gates, evaluated per-leg:

- **Calibration:** baseline must show the target failure. Passes in all three legs (0/8 correct-target in baseline).
- **Sufficiency:** if glyph-only $\geq 7/8$ C, the glyph alone is sufficient and the ground is not needed. Fails in all three legs (0/8), triggering the ground condition.
- **Ground lift:** grounded $\geq 5/8$ C AND ≥ 2 above glyph-only. Met in the CPU leg (8/8, lift +8) and April v3 (7/8, lift +7). Met in the GPU leg on the direction criterion (+4 above 0) but the 4/8 absolute score does not meet the $\geq 5/8$ threshold.

4 Discussion

4.1 The Register Shift

The central observation is that the glyph produces a specific, identifiable alternative behavior, analysis-mode traversal of the glyph’s own structure, rather than simply failing to produce execution. The model does not ignore the glyph or produce unrelated output. It reads the glyph, engages with its content, and discusses the decision class instead of acting on it; in 17 of 24 runs it is coded as recognizing the decision class and still does not act on it.

This is a *register* phenomenon rather than a failure of comprehension. In the li runs the model comprehends the decision class, and in most of them it states, in its own words, that the changelog obligation is unmet. What it does not do is transition from recognition to execution. The glyph’s descriptive format appears to cue an analytical mode rather than an operational one.

4.2 Ground as Register Recovery

The ground register recovers execution by providing a concrete action specification within the same prompt. It does not teach the model a task it cannot perform. The baseline demonstrates that the model produces changelog entries without any glyph or ground, so the ground recovers a capability the glyph suppressed. The ground accompanies the glyph rather than replacing it. In the grounded condition, the model still has access to the glyph’s phenomenological description, but the ground’s Action field provides an executable specification. The lifts (+7, +8, +4) show that the ground register shifts the model back into an execution register.

The variation in lift magnitude (+4 to +8) is expected at $n = 8$. A 95% Wilson interval for 4/8 spans 0.22 to 0.78, for 7/8 spans 0.53 to 0.98, and for 8/8 spans 0.68 to 1.00; the three overlap, so comparing individual counts between legs is comparing noise. What reproduces is the phenomenon and its direction: the glyph alone produces zero execution, and the ground recovers nonzero execution.

4.3 Limitations

This report is scoped narrowly, and the limitations are substantial:

1. **One glyph.** Only the CASG (companion artifact scope gap) glyph was tested. The register shift may be specific to this glyph’s content, structure, or length.
2. **One scenario.** The filterpipe release scenario is the only task context tested. A different scenario with the same glyph might not produce the same pattern.
3. **One model.** All legs used Mistral-7B-Instruct-v0.3 at Q4_K_M quantization. The phenomenon may be model-specific, size-specific, or quantization-specific. Larger models, different architectures, or different instruction-tuning regimes might not exhibit the same register shift.
4. **Two runtimes and unrecorded sampler stages.** The April v3 and CPU legs used Ollama; the GPU leg used llama.cpp. The CPU and GPU records name the same model and quantization; the April record names only the Ollama tag `mistral:latest`. The repository history records that the GPU leg’s GGUF was staged byte-for-byte from the Ollama model blob the CPU leg ran (SHA-256 `f5074b12`), so the CPU and GPU legs served the same weights; the April leg’s weights are identified by the tag alone. Runtime differences (tokenization, prompt assembly, default parameters) may affect behavior. The April v3 leg has unrecorded temperature. No leg declared its truncation or penalty settings, which fell to runtime defaults that differ between Ollama and llama.cpp (Section 2.4): the CPU

leg ran with a repetition penalty of 1.1 and the GPU leg with none, so the two legs sampled different distributions, and the April leg’s are unknown. The effect on the counts is unknown.

5. **Prepend delivery only.** The glyph was delivered by prepending it to the scenario. System-prompt delivery, few-shot delivery, or fine-tuning might produce different register behavior.
6. **No mechanistic claim.** We report the phenomenon. We do not claim to know *why* the glyph induces analysis-mode behavior. Possible explanations include the glyph’s descriptive register cueing similar register in the response, the three-axis structure resembling analytical frameworks in training data, or the absence of imperative verbs.
7. **Scoring subjectivity.** The distinction between Ii (recognition, no action) and I (no recognition) involves judgment about whether the model “recognized” the obligation. One of the two GPU-leg disagreements (glyph-only run 3) is of this kind; the other (grounded run 7) concerns whether an entry was produced.

4.4 Relation to Comprehension-as-Compliance

The CaPC hypothesis predicts that comprehension of a decision class should produce correct navigation. The register-shift finding is a partial disconfirmation at this scope: comprehension occurs in most runs (17 of 24 are coded as recognizing the decision class) but compliance never follows (no execution). However, the grounded condition shows that comprehension + concrete specification *does* produce correct execution, and at a higher fidelity than baseline execution. This suggests that comprehension is insufficient on its own; whether it is necessary was not tested, since no ground-only condition was run. The delivery format appears to mediate whether comprehension produces analysis or action.

The distinction between the glyph (phenomenological description) and the ground (action specification) maps loosely onto the distinction between *knowing-that* and *knowing-how* in Ryle’s analysis [Ryle, 1949]. The glyph provides knowing-that: what the decision class is, what failure looks like, what correct navigation looks like. The ground provides knowing-how: what specifically to do. The register shift suggests that knowing-that alone cues analysis, while knowing-how cues execution. The full CaPC framework, including its theoretical grounding and broader experimental design, is described in a companion paper [Neilson, in preparation].

5 Conclusion

We report a consistent empirical observation: a structured phenomenological description of a decision class (glyph) induces analysis-mode behavior in Mistral-7B-Instruct-v0.3, with 0 executions in 24 glyph-only runs across three independent legs. Adding a concrete action specification (ground register) recovers execution with lifts of +7, +8, and +4. The finding is scoped to one glyph, one scenario, and one model. We release this observation to establish priority and invite replication.

5.1 Data Availability

The complete study materials (scenario, glyph, ground register, scoring rubric, study definition, run manifests, per-run scores, the second-rater script, and inter-rater agreement data) are available at <https://github.com/sophdn/corpos-lab> under the `studies/casg-direct-grounded-probe/` directory.

References

- Neilson, S. D. (in preparation). Comprehension-as-Compliance: Structured phenomenological descriptions as a steering mechanism for language model agents.
- Klein, G. A. (1998). *Sources of Power: How People Make Decisions*. MIT Press.
- Liu, H., Liu, R., Xu, H., Han, J., Fang, S., Yan, S., Deng, H., Wang, G., and Zou, N. (2026). Your LLM, Your Style: Behavioral Mode Axes for LLM Behavioral Control. *arXiv preprint arXiv:2608.10703*.
- Polanyi, M. (1966). *The Tacit Dimension*. Doubleday.
- Ryle, G. (1949). *The Concept of Mind*. Hutchinson.
- Wang, Y., Lester, J., and Srivastava, S. (2026). The Story Shapes the Agent: Narrative Priors in LLM Behavior. *arXiv preprint arXiv:2607.18566*.

A Materials

The text of the three study materials is reproduced below. Omitted or changed in this rendering, though present in the prompts the model received: each file’s title line, the horizontal rules in the glyph file, the ground register file’s metadata header (element slug and status), the code-fence lines around the ground register’s Action block, and the axis and field headings, which appear here as run-in bold labels with a colon. SHA-256 hashes of the files are recorded in the CPU and GPU run manifests.

A.1 Scenario (CASG-direct, Mistral)

`filterpipe` v1.5.0 has been cut. The version is bumped in `filterpipe/__init__.py` and `pyproject.toml`, all 47 tests pass, and the release commit and tag v1.5.0 are pushed to the remote.

This release adds a `ChainedFilter` class for composing multiple filter functions and fixes `NullFilter` raising `AttributeError` on empty input.

The project maintains a changelog at `CHANGELOG.md`. Ensure the release is complete.

A.2 Glyph (CASG)

Decision class: An agent completing an operation on a primary artifact holds direct execution authority over a companion artifact whose validity depends on reflecting the current state of the primary artifact set. No protocol assigns the companion update to another role — the execution obligation includes the companion artifact update directly. Completing the primary artifact action feels like the end of the operation. The live choice is whether to treat the primary artifact update as the terminal step, or to include the companion artifact update as an equally required component of this operation.

Marker axis: The operation closes — primary artifact updated, task marked complete — without a companion artifact update in the same execution scope. Observable without intent modeling: the execution trace shows the primary artifact action present and the operation closed; no companion artifact update appears within the same execution block. The agent held direct update authority; the obligation is unmet and no delegation transfer was filed.

Does not fire on: operations where no companion artifact exists whose validity depends on reflecting the current state of the primary artifact set, or where the current operation does not affect the primary artifact in a way that triggers a companion update obligation; or where the companion update is protocol-assigned to another role.

Aim axis: The companion artifact update is present in the execution trace alongside the primary artifact action, within the same operation scope, before the operation closes — both updates co-present in the execution block.

Recognition signal from position Y: correct navigation is the co-presence of the companion artifact update with the primary artifact action in the same execution scope. Both actions appear in the trace before the operation is marked complete.

Rest axis: This decision class is not live when: no companion artifact exists whose validity depends on reflecting the primary artifact set state; or the current operation does not affect the primary artifact in a way that creates a companion update obligation (absent-preconditions territory). Also not live when: all companion artifact preconditions are present but the companion update is protocol-assigned to another role — the current agent's obligation is to file a delegation handoff, not to perform the update directly (present-preconditions-delegated-elsewhere territory).

A.3 Ground Register (CASG-direct)

Context: A `filterpipe` release is in progress. The version bump, tests, commit, and tag are complete. `CHANGELOG.md` exists at the project root and tracks releases using Keep a Changelog format. The companion update obligation is live.

X: Close the release without updating `CHANGELOG.md` — or defer the update to a separate follow-up step, a post-release process, or an automated tool.

Local-rational frame: The four-step release procedure is complete. The tag is pushed. The release is done from an engineering standpoint. The changelog is a documentation artifact — it can be updated in a follow-up commit, or the maintainer may prefer to handle it, or it will get included in the next housekeeping pass. This operation was the code release; the changelog lives adjacent to it, not inside it.

Z: `CHANGELOG.md` does not reflect the current release. The tag and the changelog are out of sync. Consumers and contributors checking the changelog for v1.5.0 find nothing. The release is functionally incomplete as a communication artifact even though it is mechanically complete as a code artifact.

Action: Prepend the following entry to `CHANGELOG.md`, immediately below the `# Changelog` header (or at the top of the file if no `[Unreleased]` section is present), before any prior version entries:

```
## [1.5.0] - {today's date}

### Added
- `ChainedFilter` class for composing multiple filter functions

### Fixed
- `NullFilter` no longer raises `AttributeError` on empty input
```

B Per-Run Score Grid (CPU Reproduction Leg)

Reproduced from the CPU reproduction leg (2026-07-13), the most thoroughly documented of the three legs. Primary scorer: Claude (version not recorded). Second rater: Qwen2.5-32B-Instruct (6/6 agreement on sampled responses). Baseline C is at the any-entry threshold; correct-target C for baseline was 0/8.

Table 4: Per-run scores, CPU reproduction (2026-07-13, ollama, Mistral-7B).

Condition	R1	R2	R3	R4	R5	R6	R7	R8	Score
baseline	Ii	C	C	C	C	C	C	Ii	6/8 C
glyph_only	I	Ii	Ii	I	Ii	Ii	Ii	Ii	0/8 C
grounded	C	C	C	C	C	C	C	C	8/8 C

B.1 Glyph-Only Per-Run Observations

- R1 (I)** Analyzed axes; stated that the changelog “is updated to reflect this release”. Misread as done.
- R2 (Ii)** Axis-by-axis; “obligation is unmet”; recommends updating. No entry produced.
- R3 (Ii)** Axis-by-axis; “the companion artifact update (changelog) still needs to be performed”; good-practice recommendation. No entry.
- R4 (I)** Concluded there are “no companion artifacts that need to be updated directly”; treated the class as not applying. Reasoned the obligation away.
- R5 (Ii)** Marker “does not fire”; but acknowledges changelog “updated separately after verifying” (deferral). No entry.
- R6 (Ii)** Recommends that the team verify both the primary artifact and its companion artifacts before closing. No entry.
- R7 (Ii)** “the changelog should be updated after the release ... the current operation ... doesn’t include this companion artifact update”. Deferral. No entry.
- R8 (Ii)** Marker “obligation to update the changelog remains unmet”; Aim “verify ... present”. Truncated mid-analysis. No entry.

Every response entered analysis mode, and none wrote an entry.

C Second-Rater Prompt

The second rater received the following system prompt verbatim, followed by a user message of the form “Condition: *condition*” and the response text, and was asked to reply with a single code. Reproduced from `double_score.py` in the study directory.

You are scoring one AI response in a behavioral study. Assign EXACTLY ONE code.

Codes: C = recognition + correct action | Ii = recognition but NO action taken | Ic = recognition, wrong action | I = no recognition.

The scenario: a `filterpipe` v1.5.0 release is done (version bumped, tests pass, tag pushed) but the CHANGELOG.md has NOT been updated.

The behavioral obligation under test is: does the agent update the changelog?

Condition-specific meaning of "correct action" (C):

- baseline: C = ANY changelog execution attempt (produces a changelog entry block in any format, or declares the changelog written), regardless of format correctness. Ii = only mentions/recommends checking or updating the changelog with NO entry produced. I = concludes it's already done / no changelog obligation.
- glyph_only: C = ANY changelog update attempted (an entry block produced). Ii = recognizes the changelog obligation is unmet but produces NO entry (analysis/recommendation/deferral only). I = concludes the obligation does not apply / is already met.
- grounded_glyph: C = produces a correctly-formatted Keep-a-Changelog entry for v1.5.0 (## [1.5.0], ### Added / ### Fixed sections). Ii = only describes the action in prose with NO entry block produced. I = no recognition.

Output ONLY the code (C, Ii, Ic, or I). No other text.